

BUILD_SCRIPT.md: Document-as-System Abstractions for AI-Mediated Programming

Stefano Casafranca Laos
Independent Researcher
Austin, Texas, USA
scasafranca101@gmail.com

Abstract

AI-supported programming tools are increasingly grounded in conversational prompting, but as interaction histories develop the design rationale that trailblazed them is becoming fragmented—that information rots within chat logs to request mechanically rather than gestalt-ed, summarized lossily by context management and distanced from the code it generated. While version history serves to capture what changed, it does not record why: the constraints, trade-offs, and intent that influenced each decision.

This paper introduces BUILD_SCRIPT.md, a document-as-system prototype where executable specifications in terms of editable mark-down documents drive AI code generation. Instead of repeating prompts from a linear history, document state is recomputed involving the whole document—so you can edit non-destructively at any cursor position without breaking later work. Enforcement hooks synchronize between document intent and generated code. We evaluate this method in a within-subjects study of 12 individual sessions (each consisting of 90 minutes) where participants completed identical development tasks using both conversational prompting and BUILD_SCRIPT.md document-driven workflows. Participants commented on the ease of interpretability of system behavior (mean 4.4/5), recovering user intent and future replicability (4.3/5), and perceived control (4.5/5) which was summarized as "...being able to read the system as a whole" rather than "assembling meaning from scattered parts of conversation". We posit that document-as-system abstractions provide a natural basis for human-centered AI programming, by shifting personalization and user modeling from ephemeral conversational state to persistent, inspectable artifacts.

CCS Concepts

• **Human-centered computing** → **User studies**; *Graphical user interfaces*.

Keywords

AI-assisted programming, Document-as-system, End-user programming, User modeling, Conversational interfaces, Human-AI interaction

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CUI '26, Bremen, Germany

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

ACM Reference Format:

Stefano Casafranca Laos. 2026. BUILD_SCRIPT.md: Document-as-System Abstractions for AI-Mediated Programming. In *Proceedings of ACM CUI 2026*. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

AI-assisted programming tools have fundamentally changed how people build software. GitHub Copilot [6] provides inline code completion, ChatGPT [16] and Claude [1] support open-ended conversational coding, and editor-integrated tools such as Cursor embed AI directly into development workflows. Across these tools, conversational prompting has become the dominant interaction paradigm: users describe intent in natural language, the AI generates code, and users refine through follow-up messages. This model lowers the barrier to programming and accelerates prototyping, but it introduces a structural problem that compounds over time.

As projects grow beyond initial prototyping, the conversational history becomes an increasingly unreliable record of design intent. Design decisions fragment across dozens or hundreds of chat messages. Constraints articulated in early prompts are buried under subsequent refinements. Context windows force lossy summarization, discarding the reasoning behind choices that the generated code still depends upon. Unlike traditional programming where code serves as the persistent artifact and documentation accompanies it, AI-assisted development inverts this relationship: the conversation is ephemeral while the generated code persists—disconnecting the “why” from the “what.”

Consider a developer who, twenty prompts into building a web application, needs to change an early decision—say, switching from button-based task completion to checkbox-based completion. In a chat-based system, they face three unsatisfying options: scroll back through the history to find and re-articulate the original constraint, hoping the AI reconstructs the full context; issue a new prompt that may contradict earlier decisions the AI has already forgotten; or manually edit the generated code, breaking synchronization between the AI’s understanding and the actual codebase. Each option fragments intent further. This tension between iteration and coherence is not a limitation of any particular model but a structural property of linear conversational interfaces applied to evolving systems.

Existing approaches address parts of this problem in isolation. Version control systems track what changed in code but not the intent behind changes. Chat logs preserve the sequence of prompts but are linear and difficult to revise non-destructively. Computational notebooks [7, 9] blend narrative and code but were not

designed for AI-mediated generation. End-user programming environments [15] provide structured representations but do not scale to complex software projects. What is needed is a representation that is simultaneously editable (like a document), executable (like a specification), and comprehensible (like a summary).

This paper introduces BUILD_SCRIPT.md, a document-as-system prototype in which editable markdown documents function as executable specifications for AI code generation. The document is not a rigid upfront plan but a living artifact that can be edited non-destructively at any point. The system recomputes generated code from the complete document state rather than replaying a linear prompt history. Enforcement hooks implemented as a plugin for Claude Code ensure the codebase remains synchronized with documented intent. The system supports three connected modes of operation: traditional prompting with invisible specification tracking, external document editing via Google Docs with automatic code synchronization, and same-session autonomous processing where remote document edits are injected directly into the active AI conversation.

We validate this approach through a within-subjects study (N=12, 90-minute sessions) comparing BUILD_SCRIPT.md workflows to chat-based prompting on matched development tasks. Participants ranged from students with light technical backgrounds to professional developers. We measured interpretability, recoverability, and perceived control through Likert scales and semi-structured interviews.

Participants rated document-based workflows significantly higher on interpretability (mean 4.4/5 vs. 3.2/5), recoverability (4.3/5 vs. 2.9/5), and perceived control (4.5/5 vs. 3.4/5). Qualitative analysis revealed that participants valued the ability to “read the system as a whole” rather than reconstruct intent from scattered conversation fragments. However, participants noted that document-based approaches require more upfront structure and are less suited to purely exploratory tasks.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the conceptual framework. Section 4 describes the system design. Section 5 details the user study methodology. Section 6 presents results. Section 7 discusses implications and limitations. Section 8 concludes.

2 Related Work

2.1 End-User Programming and Computational Documents

End-user programming research has long explored representations that embed computation within familiar artifacts. Spreadsheets [15] remain the most successful example: the grid simultaneously serves as the interface, the program, and the output. Users see what computes, and they manipulate computation by manipulating data. This directness is what makes spreadsheets accessible, though they struggle with abstraction and composition at scale.

Computational notebooks such as Jupyter and Observable extended this idea by interleaving narrative text, executable code, and output within a single document [9]. Notebooks succeed at supporting exploratory data analysis but introduce structural problems when used for persistent software development. Execution order becomes ambiguous as cells are re-run out of sequence, hidden

state accumulates, and the narrative structure fragments as projects grow [7]. Knuth’s literate programming [11] anticipated the value of co-locating prose and code, but assumed human authorship of both. None of these systems were designed for AI-mediated code generation, where the document’s role shifts from documenting code to *specifying* it.

Recent work on collaborative computational documents addresses synchronization challenges. Local-first software [10] advocates for designs where users own their data and collaboration does not require centralized servers. CRDTs for rich text editing [12] enable concurrent authoring with automatic conflict resolution. These systems solve the problem of multiple humans editing the same document but do not address the problem of maintaining synchronization between a human-authored specification and AI-generated code.

2.2 Live Programming and Provenance

Live programming systems [14] tighten the feedback loop between code modification and observable behavior, making computation more visible during development. Edwards’ Subtext [5] demonstrated that programming could be simplified by treating programs as live, inspectable artifacts rather than static text. Petricek’s work on live data exploration [17] enables non-linear interaction with computational artifacts, allowing users to revisit and modify earlier computations without replaying the full history. This is conceptually similar to our non-destructive editing model but applied to data transformations rather than code generation.

Provenance tracking [4] records the chain of transformations that produced a given result, preserving the “why” behind computed outputs. This is analogous to our goal of preserving the “why” behind generated code. However, provenance systems operate at the level of data transformations, while we operate at the level of *intent*—tracking why a design decision was made, not how data flowed through a pipeline. The gap between data provenance and intent provenance is precisely where document-as-system abstractions contribute.

2.3 AI-Assisted Programming and Conversational Coding

The landscape of AI-assisted programming tools can be organized by how they represent user intent. Autocomplete tools like GitHub Copilot [6] infer intent from surrounding code context, requiring no explicit specification [2]. Conversational tools like ChatGPT [16] and Claude [1] accept natural language prompts but store intent only in the linear chat history [18]. Editor-integrated tools embed AI within development environments but still rely on ad-hoc prompting. Studies of these tools reveal consistent usability challenges: users struggle to verify generated code against their original intent [20], and the gap between natural language specification and generated output creates friction [13].

Sarkar and Drosos [19] introduced the concept of *Vibe Coding*, describing how developers use conversational AI for iterative, exploratory programming. Their key finding—that developers struggle with intent management as conversations grow—directly motivates our document-based approach. When the accumulation of prompts outpaces the developer’s ability to maintain a coherent

mental model, the conversation ceases to be a reliable interface for managing system behavior.

The gap across these tools is consistent: existing systems treat conversation as ephemeral or treat code as the only persistent artifact. No current system provides a persistent, editable representation of intent that remains synchronized with generated code.

2.4 Technical Dimensions of Programming Systems

Jakubovic et al. [8] proposed a framework of *technical dimensions* for analyzing programming systems, including notation, abstraction mechanisms, feedback loops, and error handling. We position BUILD_SCRIPT.md along several of these dimensions: its notation is structured markdown (accessible to non-programmers), its abstraction is the document-as-system paradigm (intent expressed as specification, not code), its synchronization mechanism uses enforcement hooks (automated validation), and its feedback loop supports incremental regeneration (targeted code updates in response to document edits).

Literate programming [11] and Org-mode share the principle of co-locating intent and computation, but they treat the document as documentation of code. BUILD_SCRIPT.md inverts this relationship: the document is the source of truth for AI-generated code. The distinction is consequential—in literate programming, the human writes both prose and code; in document-as-system, the human writes prose and the AI generates code, making the document the primary artifact of authorship and control.

3 Conceptual Framework

3.1 The Document-as-System Abstraction

We define *document-as-system* as an abstraction in which system behavior emerges from transformations of a structured document rather than from replaying a temporal interaction log. Prompts, constraints, and refinements are treated as persistent document elements that collectively define program behavior. The document is not merely documentation of the system, nor a plan to be executed once. Instead, it is a living specification that continuously governs system behavior. Changes to the document trigger regeneration of affected code. The document is editable at any point—not just append-only like chat.

This framing contrasts with established paradigms along a key axis. In traditional programming, code *is* the system. In literate programming, code and documentation coexist within the same artifact. In conversational AI, chat is the interaction interface but not the persistent representation. In document-as-system, the document is the system’s intent, and code is derived from it. The document functions simultaneously as the source of truth and an executable specification, while AI systems act as collaborative interpreters that translate user intent into behavior.

This abstraction enables four properties that chat-based interfaces lack:

- (1) **Scannable:** The entire specification can be read top-to-bottom as a coherent description of the system.
- (2) **Editable:** Any part can be revised without invalidating the whole. Editing an early constraint does not require re-executing all subsequent prompts.

Table 1: Comparison of programming paradigms by representation and execution model.

Dimension	Chat-Based	Document-Based
Representation	Linear conversation	Structured document
Persistence	Ephemeral (chat log)	Persistent (markdown)
Editing	Append-only prompts	Non-destructive edits
Synchronization	Replay or summarize	Recompute from state
Scalability	Degrades with length	Constant (scannable)

- (3) **Executable:** Changes trigger concrete system behavior—AI code generation, validation, and deployment.
- (4) **Inspectable:** The relationship between intent and generated code is transparent and traceable.

3.2 BUILD_SCRIPT.md as Executable Specification

BUILD_SCRIPT.md instantiates the document-as-system abstraction as a markdown file with a specific structure. The primary section, “Prompts Up to Date with Output,” contains a continuous paragraph summarizing the current project state in natural language. Unlike enumerated prompt lists, this paragraph reads as a coherent specification that can be edited at any point:

```
Build a task manager web app with React and
TypeScript showing a form to add tasks with titles,
a list of tasks with checkboxes for completion
using reduced opacity and a green checkmark icon,
and filter buttons for All, Active, and Completed
views using local storage for persistence.
```

The AI reads the complete document, interprets the paragraph as a set of requirements, and generates or updates code accordingly. When the user edits a phrase—for example, changing “checkboxes for completion” to “swipe gestures for completion”—the system identifies the modification through sentence-level diffing and regenerates only the affected code. This means editing an early constraint does not require re-executing all subsequent prompts; the system recomputes from the revised document state.

3.3 Comparison with Chat-Based Paradigm

Table 1 contrasts the two paradigms across five dimensions. The fundamental difference is in how change is expressed: chat-based systems bind meaning to interaction order (modifying an early prompt requires restoring a checkpoint or manually reconstructing subsequent prompts), while document-based systems bind meaning to document structure, allowing edits at any point.

These paradigms are not mutually exclusive. Chat is more natural for exploration and discovery—when the user does not yet know what they want, conversational back-and-forth supports incremental refinement. Documents require more upfront structure but scale better as projects grow. We return to this complementarity in Section 7.

4 System Design

4.1 Architecture Overview

BUILD_SCRIPT.md is implemented as a plugin for Claude Code, a command-line AI programming tool. The plugin operates through three enforcement hooks that fire automatically during the development workflow, requiring no explicit user action:

- (1) **UserPromptSubmit hook:** Fires on every user message. Injects invisible instructions that direct the AI to maintain BUILD_SCRIPT.md as a continuous-paragraph specification, updating it to reflect the current project state after each interaction.
- (2) **PostToolUse hook:** Fires after every file write or edit. Automatically commits changes to version control with a tagged commit message, providing a complete audit trail of both code and specification evolution.
- (3) **Stop hook:** Fires when the AI finishes responding. Checks for queued changes from external document edits (e.g., Google Docs) and injects them into the active session if present.

The system supports three connected modes of operation. In **Claude Code Chat** mode, users prompt the AI conversationally while hooks silently maintain the specification. In **Google Docs** mode, users edit a synced copy of BUILD_SCRIPT.md from any device, and changes propagate to the local file. In **Same-Session Autonomous** mode, remote document edits are detected by a polling daemon, differenced at the sentence level, and injected directly into the active AI conversation—enabling development from a phone while the AI generates code on a laptop with full conversation context.

The system is framework-agnostic: it auto-detects the project type (React, Vue, Next.js, Django, Flask, etc.) from project files and adapts its behavior accordingly.

4.2 Document Structure

The specification uses a structured markdown format with two files:

BUILD_SCRIPT.md contains the living specification. Its primary section, “Prompts Up to Date with Output,” is a continuous natural-language paragraph describing the current system. Additional sections capture project metadata (name, tech stack, directory structure) and available commands. This file is the bidirectional sync target—it is both written by the AI (reflecting what was built) and readable by the user (as a specification of the system).

BUILD_SCRIPT_FULL.md extends this with an append-only “Prompts RAW” section that records every original prompt verbatim, including those whose intent was later revised. This provides full provenance: the continuous paragraph captures *current* intent, while the raw history captures the *evolution* of intent.

4.3 Enforcement Hooks

The UserPromptSubmit hook operates invisibly. It prepends instructions to every user message directing the AI to maintain BUILD_SCRIPT.md. These instructions specify that the document should be *edited* (not appended to) when requirements change—if

the user says “use checkboxes instead of buttons,” the existing description of buttons is replaced, not supplemented. This ensures the document always reads as a coherent current-state description.

The PostToolUse hook fires after every file modification, executing `git add` and `git commit` with a tagged message. This creates a fine-grained version history where each commit captures both the code change and any corresponding specification change, enabling researchers and developers to trace how intent and implementation co-evolved.

4.4 Bidirectional Synchronization

A standalone Node.js daemon (`sync-gdoc.js`) polls a linked Google Doc every second, comparing content hashes of the local BUILD_SCRIPT.md and the remote document. When the remote document changes (indicating the user edited it from another device), the daemon waits for 10 seconds of editing silence (a debounce window to allow the user to finish their thought), then computes a sentence-level diff using Longest Common Subsequence (LCS) alignment.

The diff classifies changes as ADDED (new sentences), REMOVED (deleted sentences), or MODIFIED (sentences with >80% similarity but not identical). This diff is written to an HMAC-signed queue file that the Stop hook or UserPromptSubmit hook consumes, injecting the changes into the active AI conversation as a structured prompt that specifies only the delta—not the full specification.

4.5 Security

Because the synchronization mechanism enables external content to be injected into an active AI session, the system implements several security measures: HMAC-SHA256 signed queue files with a shared secret, content sanitization (stripping shell metacharacters, control characters, and dangerous HTML), rate limiting (minimum 30 seconds between injections), nonce replay protection (preventing reuse of consumed queue files), timestamp validation (rejecting queues older than 30 minutes), and restrictive file permissions (mode 0600) on all security-sensitive files.

5 User Study

5.1 Research Questions

We designed a within-subjects study to answer the following research questions:

- RQ1:** How does document-based specification compare to chat-based prompting in terms of *interpretability* of system behavior?
- RQ2:** How do the two approaches differ in *recoverability* of user intent after multiple iterations?
- RQ3:** What is the perceived *sense of control* in each paradigm?
- RQ4:** What interaction frictions and affordances emerge in each approach?

5.2 Participants

We recruited 12 participants through university mailing lists and professional networks. Inclusion criteria required at least a light technical background (able to read code and use a command-line tool) but not necessarily professional development experience. This targets the “power user” population that AI-assisted programming

Table 2: Participant demographics (N=12).

Characteristic	Category	Count
Gender	Male	7
	Female	5
Technical Background	Student (CS/related)	4
	Professional Developer	5
	Researcher/Academic	3
Prior AI Tool Use	None	2
	Occasional (ChatGPT, Copilot)	6
	Frequent (daily use)	4
Self-Rated Programming Skill (1–5 scale)	Beginner (1–2)	3
	Intermediate (3)	4
	Advanced (4–5)	5

tools are increasingly designed for—people who are technically literate but may not write code as their primary activity.

Table 2 summarizes participant demographics. Participants ranged from computer science students to professional developers and academic researchers, with varying levels of prior experience with AI-assisted tools.

5.3 Study Design

Each participant completed the same development task twice—once using Claude Code in its standard chat-based mode (control condition) and once using Claude Code with the BUILD_SCRIPT.md plugin active (experimental condition). Condition order was counterbalanced across participants to control for learning effects: six participants completed the chat condition first, six completed the document condition first.

In both conditions, participants followed an identical five-step prompt sequence to build a task manager web application. The prompt sequence was designed to exercise iterative refinement and intent revision:

- (1) Create a task manager app with React and TypeScript using Vite.
- (2) Add the ability to add tasks, mark them complete (with strikethrough text), and delete them.
- (3) *Change of mind*: Replace strikethrough with reduced opacity (0.6) and a green checkmark icon.
- (4) Add filter buttons for All, Active, and Completed tasks.
- (5) Add due dates with a date picker for each task.

Prompt 3 is critical: it requires the system to revise an earlier design decision. In the control condition, the chat history contains both the original instruction (strikethrough) and the revision (opacity + checkmark), requiring the AI to reconcile the contradiction. In the experimental condition, the BUILD_SCRIPT.md plugin edits the specification in place, so the document always reflects only the current intent.

In the experimental condition, the plugin hooks operated silently—participants were not told that BUILD_SCRIPT.md was being maintained during task execution. After completing the task, participants in the experimental condition were shown the generated BUILD_SCRIPT.md and asked to compare it with their chat history from the control condition. User-authored prompts were matched

across conditions, while system-level instructions differed by condition design.

5.4 Procedure

Each 90-minute session followed this timeline:

- (1) **Introduction**: Informed consent, demographic questionnaire.
- (2) **Training**: Tutorial on the first assigned tool, including a warm-up task (building a simple counter app).
- (3) **Task 1**: Complete the task manager using the first tool. The researcher observed and took notes but did not intervene unless a participant was stuck for more than 5 minutes.
- (4) **Questionnaire 1**: Likert scales for interpretability, recoverability, control, satisfaction, and efficiency.
- (5) **Break**.
- (6) **Training 2**: Tutorial and warm-up for the second tool.
- (7) **Task 2**: Complete the same task manager using the second tool.
- (8) **Questionnaire 2**: Same Likert scales.
- (9) **Semi-structured interview**: Open-ended questions comparing both approaches.

Participants provided informed consent, could withdraw at any time, and were debriefed after the session, including disclosure of the silent BUILD_SCRIPT.md maintenance in the experimental condition.

5.5 Measures

Quantitative. Five Likert-scale items (1=Strongly Disagree, 5=Strongly Agree) were administered after each condition:

- *Interpretability*: “I could easily understand what the system was doing and why.”
- *Recoverability*: “I could easily find and revise earlier decisions without breaking the system.”
- *Control*: “I felt in control of the system’s behavior.”
- *Satisfaction*: “I was satisfied with the tool for this task.”
- *Efficiency*: “I could complete the task efficiently without wasting time.”

Qualitative. Semi-structured interviews explored preferences, pain points, and perceived tradeoffs. Questions included: “Which approach did you prefer and why?”, “Describe a moment when you felt frustrated or confused”, “How did you keep track of what the system had done?”, and “If you had to extend this project further, which tool would you choose?” Interviews were audio-recorded, transcribed, and analyzed using reflexive thematic analysis [3]. Two researchers independently coded transcripts, iteratively refined codes through discussion, and organized them into themes.

6 Results

6.1 Quantitative Findings

Table 3 presents subjective ratings across both conditions. Participants rated the BUILD_SCRIPT.md condition higher on all measures except efficiency, where ratings were comparable.

Paired-samples *t*-tests revealed statistically significant differences between conditions for Interpretability ($t(11) = 5.23, p < .001, d = 1.51$), Recoverability ($t(11) = 4.82, p < .001, d = 1.39$),

Table 3: Subjective ratings by condition (1=Strongly Disagree, 5=Strongly Agree, N=12).

Measure	Chat-Based			BUILD_SCRIPT.md		
	Mean	SD	Range	Mean	SD	Range
Interpretability	3.2	0.8	2–5	4.4	0.5	4–5
Recoverability	2.9	1.0	1–4	4.3	0.7	3–5
Control	3.4	0.9	2–5	4.5	0.5	4–5
Satisfaction	3.5	0.7	2–5	4.2	0.6	3–5
Efficiency	3.8	0.6	3–5	3.9	0.8	2–5

Control ($t(11) = 4.47, p < .001, d = 1.29$), and Satisfaction ($t(11) = 3.15, p < .01, d = 0.91$). The difference in Efficiency was not statistically significant ($t(11) = 0.41, p = .69, d = 0.12$), indicating that the document-based approach does not impose meaningful time costs compared to chat-based prompting. Effect sizes for the first three measures were large ($d > 1.0$), suggesting the differences are practically meaningful.

Two participants rated BUILD_SCRIPT.md lower on efficiency due to the upfront structuring required in the document condition. Both were frequent AI tool users (daily ChatGPT users) who reported being well-practiced at managing conversational context through prompting strategies.

6.2 Qualitative Themes

Thematic analysis of interview transcripts produced four themes, organized below by research question.

6.2.1 Theme 1: Reading vs. Searching (RQ1). The most consistent finding was that BUILD_SCRIPT.md enabled participants to *read* the system as a coherent whole, while chat required them to *search* for fragments of prior intent:

“With the document, I could just scroll and see everything at once. With chat, I had to remember where I said something or use Ctrl+F.” —P7

“The markdown file felt like a spec sheet. I knew where to look for each feature. The chat was like a diary—chronological but not organized.” —P3

Nine of twelve participants spontaneously used spatial or visual metaphors to describe their experience with the document (“I could see the whole picture,” “everything was laid out”), while descriptions of chat were temporal (“I had to go back,” “I couldn’t find where I said that”). This suggests that the scannable structure of documents engages a different cognitive mode than the sequential retrieval required by chat logs.

6.2.2 Theme 2: Edit vs. Append (RQ2). Participants approached revisions fundamentally differently across conditions. In chat, they issued corrective prompts (“Actually, change the button to a checkbox”), which sometimes contradicted earlier instructions still present in the conversation history. In the document condition, the specification was edited in place, maintaining a consistent current state:

“I could just go back and fix the description, and the system updated the code. In chat, I had to say it again and hope it remembered the context.” —P11

Table 4: Observed interaction frictions (N=12 sessions per condition).

Friction Type	Chat	BUILD_SCRIPT.md
Context loss (earlier decisions)	Frequent	Rare
Conflicting instructions	Occasional	Rare
Difficulty locating prior intent	Frequent	Rare
Sync errors (code vs. intent)	Occasional	Occasional
Upfront structuring burden	N/A	Occasional
Learning curve (tool-specific)	Rare	Occasional

However, three participants initially tried to *append* to the document (adding new sections at the bottom) rather than editing existing text in place. This suggests that the affordance of non-destructive editing is not immediately intuitive to users accustomed to append-only conversational interfaces—explicit guidance may be needed.

6.2.3 Theme 3: Control vs. Surprise (RQ3). Participants described qualitatively different relationships with the AI across conditions. With BUILD_SCRIPT.md, participants felt they *authored* the system by writing its specification. With chat, they felt they were *negotiating* with the AI—making requests without certainty about the outcome:

“The document made me feel like I was designing the system. The chat made me feel like I was asking for favors.” —P5

This aligns with the significantly higher control ratings in the document condition. At the same time, some participants valued chat’s conversational flexibility for exploratory tasks:

“When I wasn’t sure what I wanted, chat was easier because I could just describe it vaguely and refine.” —P9

This suggests the two paradigms may be complementary rather than competitive, each suited to different phases of development.

6.2.4 Theme 4: Upfront Structure vs. Incremental Discovery (RQ4). BUILD_SCRIPT.md requires articulating structure (sections, constraints, architectural decisions) that chat-based interaction discovers incrementally. Several participants found this burdensome initially but valuable as the project grew:

“At first, I didn’t want to write headings and organize everything. But by the end, I was glad I did because I could navigate easily.” —P2

This tradeoff—upfront structure vs. incremental discovery—represents a design tension inherent to document-based approaches. Chat allows users to defer decisions, while documents require explicit articulation. The cost of structuring is paid early, but the payoff in navigability increases over time.

6.3 Observed Interaction Frictions

Table 4 summarizes observed interaction frictions across both conditions, coded from researcher observation notes.

Chat-based frictions predominantly involved memory and coherence: losing track of earlier decisions, issuing contradictory instructions, and difficulty locating prior intent in long histories.

Document-based frictions involved upfront effort and learning: participants needed time to understand the document structure and the editing-vs-appending distinction. Notably, synchronization errors (cases where generated code did not match documented intent) occurred at similar rates in both conditions, suggesting this is a property of AI code generation rather than the interaction paradigm.

7 Discussion

7.1 Implications for Conversational User Interfaces

Our findings suggest that many breakdowns in AI-mediated programming stem from the *representation* of user intent rather than from model capability. Chat-based interfaces obscure accumulated intent behind linear history, while document-based abstractions preserve it as a manipulable, inspectable artifact. This has direct implications for conversational user interface design: as conversational AI systems are applied to increasingly complex, long-running tasks, the append-only structure of chat becomes a liability rather than an asset.

The document-as-system approach does not replace conversation—it complements it. Conversation excels at exploration, clarification, and rapid iteration. Documents excel at consolidation, persistence, and revision. A productive design pattern may be to start with conversational exploration and “crystallize” decisions into a document as the project stabilizes, a transition that our system supports through its invisible specification tracking.

BUILD_SCRIPT.md sits between traditional programming (full control, high expertise) and conversational AI (low barrier, limited control at scale). By providing a structured representation of intent that is both human-readable and machine-executable, it offers a middle ground: accessible enough for non-expert users but scalable enough for complex, evolving projects.

7.2 When to Use Documents vs. Chat

Based on our findings, we offer the following design guidance:

- **Chat is better for:** Exploration, rapid prototyping, one-off scripts, and tasks where the final artifact is disposable.
- **Documents are better for:** Long-lived projects, collaborative work, systems requiring traceability, and tasks where intent must be preserved and revised over time.
- **Hybrid approach:** Start with chat for exploration, then crystallize decisions into a document as the project stabilizes. Our system supports this naturally—the invisible hooks generate the document from chat interactions, and the user can take over editing when they choose.

Tools could support this transition explicitly. For example, a system could summarize key decisions from a chat history into a structured document, allowing users to start conversationally and transition to document-based management without re-articulating their intent.

7.3 Limitations

Several limitations constrain the generalizability of our findings:

- **Sample size:** $N=12$ is sufficient for within-subjects comparison and yields large effect sizes, but limits statistical power for detecting small effects and generalization to broader populations.
- **Task scope:** A single, moderately complex task (task manager app) does not capture the full range of real-world software development. Larger, messier projects may amplify or attenuate the observed differences.
- **Participant expertise:** Our sample skewed toward technically savvy users. Less technical users may struggle more with document structuring, potentially reducing the benefits we observed.
- **Tool maturity:** BUILD_SCRIPT.md is a research prototype, not a polished product. Usability friction attributable to prototype quality may have affected ratings in both directions.
- **Novelty effects:** Participants may have been favorably biased toward the document approach simply because it was novel. Longitudinal studies would be needed to distinguish novelty from sustained preference.

7.4 Future Work

Several research directions follow from this work. **Collaborative document editing** would extend BUILD_SCRIPT.md to multi-user contexts, leveraging CRDTs [12] for concurrent specification authoring. **AI-assisted document authoring** could bridge the chat-document gap by having the AI draft specification sections from conversational input, reducing the upfront structuring burden participants identified. **Provenance integration** would automatically link generated code back to document sections, allowing developers to ask “why does this code exist?” and trace to the originating specification. **Longitudinal evaluation** with larger samples and diverse tasks would test whether the benefits we observed persist over extended use. Finally, **evaluation with non-programmers** would test whether document-based specifications lower the barrier to programming for domain experts (e.g., scientists, designers) who can express computational intent in structured prose without writing code.

8 Conclusion

AI-assisted programming tools have democratized code generation but struggle with intent management as projects grow. Conversational histories fragment design decisions across linear chat logs, making it difficult to understand, revise, and replicate complex systems. Version control captures what changed in code but not the constraints, tradeoffs, and intent that shaped each decision.

We introduced BUILD_SCRIPT.md, a document-as-system prototype where editable markdown documents function as executable specifications for AI code generation. The system recomputes behavior from the complete document state rather than replaying a linear prompt history, and enforcement hooks ensure the codebase remains synchronized with documented intent. The system supports development from any device through bidirectional synchronization with Google Docs, with remote edits injected directly into the active AI conversation.

A within-subjects study ($N=12$) demonstrated that document-based approaches significantly improve interpretability ($M = 4.4$

vs. 3.2, $p < .001$), recoverability ($M = 4.3$ vs. 2.9, $p < .001$), and perceived control ($M = 4.5$ vs. 3.4, $p < .001$) compared to chat-based prompting. Participants valued the ability to “read the system as a whole” and edit specifications non-destructively, while noting that documents require more upfront structure than exploratory conversation.

As AI becomes more capable of generating code, the bottleneck shifts from writing code to expressing intent clearly and managing it over time. Document-as-system abstractions address this challenge by treating the specification as a first-class artifact—persistent, inspectable, and editable—shifting AI programming from ephemeral conversational state to artifacts that support replicability and long-term comprehension.

References

- [1] Anthropic. 2023. Claude: A Next-Generation AI Assistant. <https://www.anthropic.com/claude>. Accessed: 2026-02-23.
- [2] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1, Article 78 (2023), 27 pages. doi:10.1145/3586030
- [3] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101. doi:10.1191/1478088706qp0630a
- [4] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474. doi:10.1561/19000000006
- [5] Jonathan Edwards. 2005. Subtext: Uncovering the Simplicity of Programming. In *Proceedings of the 20th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, New York, NY, USA, 505–518. doi:10.1145/1094811.1094851
- [6] GitHub. 2021. GitHub Copilot: Your AI Pair Programmer. <https://github.com/features/copilot>. Accessed: 2026-02-23.
- [7] Andrew Head, Fred Hohman, Titus Barik, Steven M. Drucker, and Robert DeLine. 2019. Managing Messes in Computational Notebooks. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–12. doi:10.1145/3290605.3300500
- [8] Joel Jakubovic, Jonathan Edwards, and Tomas Petricek. 2023. Technical Dimensions of Programming Systems. *The Art, Science, and Engineering of Programming* 7, 3, Article 13 (2023), 59 pages. doi:10.22152/programming-journal.org/2023/7/13
- [9] Mary Beth Kery, Marissa Radensky, Mahima Arya, Bonnie E. John, and Brad A. Myers. 2018. The Story in the Notebook: Exploratory Data Science using a Literate Programming Tool. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 1–11. doi:10.1145/3173574.3173748
- [10] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. 2019. Local-First Software: You Own Your Data, in Spite of the Cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. ACM, New York, NY, USA, 154–178. doi:10.1145/3359591.3359737
- [11] Donald E. Knuth. 1992. *Literate Programming*. Center for the Study of Language and Information, Stanford, CA.
- [12] Geoffrey Litt, Sarah Lim, Martin Kleppmann, and Peter van Hardenberg. 2022. Peritext: A CRDT for Collaborative Rich Text Editing. *Proceedings of the ACM on Human-Computer Interaction* 6, CSCW2, Article 531 (2022), 27 pages. doi:10.1145/3555644
- [13] Michael Xieyang Liu, Advait Sarkar, Carina Negreanu, Benjamin Zorn, Jack Williams, Neil Toronto, and Andrew D. Gordon. 2023. “What It Wants Me To Say”: Bridging the Abstraction Gap Between End-User Programmers and Code-Generating Large Language Models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, Article 64, 31 pages. doi:10.1145/3544548.3580817
- [14] Sean McDirmid. 2013. Usable Live Programming. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software*. ACM, New York, NY, USA, 53–62. doi:10.1145/2509578.2509585
- [15] Bonnie A. Nardi and James R. Miller. 1990. The Spreadsheet Interface: A Basis for End User Programming. In *Proceedings of the IFIP TC 13 Third International Conference on Human-Computer Interaction (INTERACT '90)*. North-Holland, Amsterdam, Netherlands, 977–983.
- [16] OpenAI. 2022. ChatGPT: Optimizing Language Models for Dialogue. <https://openai.com/blog/chatgpt>. Accessed: 2026-02-23.
- [17] Tomas Petricek. 2020. Foundations of a Live Data Exploration Environment. *The Art, Science, and Engineering of Programming* 4, 3, Article 8 (2020), 37 pages. doi:10.22152/programming-journal.org/2020/4/8
- [18] Steven I. Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D. Weisz. 2023. The Programmer’s Assistant: Conversational Interaction with a Large Language Model for Software Development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. ACM, New York, NY, USA, 491–514. doi:10.1145/3581641.3584037
- [19] Advait Sarkar and Ian Drosos. 2025. Vibe Coding: Programming Through Conversation with Artificial Intelligence. <https://arxiv.org/abs/2506.23253>. doi:10.48550/arXiv.2506.23253 arXiv preprint arXiv:2506.23253. Presented at PPiG 2025.
- [20] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems Extended Abstracts*. ACM, New York, NY, USA, Article 332, 7 pages. doi:10.1145/3491101.3519665

A Sample BUILD_SCRIPT.md Document

The following is a representative BUILD_SCRIPT.md generated during the study task:

```
# BUILD_SCRIPT

## Prompts Up to date with Output
Build a task manager web app with React and TypeScript using Vite showing a form to add tasks with titles, a list displaying each task with reduced opacity (0.6) and a green checkmark icon when completed, delete buttons for each task, and filter buttons for All, Active, and Completed views. Add due dates with a date picker when creating tasks and display the due date on each task card. Use localStorage for client-side persistence with auto-save.

## Project
- **Name:** Task Manager
- **Tech Stack:** React 19, TypeScript, Vite 7
- **Structure:**
  src/
    components/
      AddTask.tsx
      TaskItem.tsx
      TaskList.tsx
      FilterBar.tsx
      App.tsx
      types.ts
- **Commands:** npm run dev (port 5173)
```

B Interview Protocol

The following semi-structured interview guide was used after both conditions:

- (1) Which approach did you prefer for this task, and why?
- (2) Describe a moment when you felt confused or frustrated. What caused it?
- (3) How did you keep track of what the system had done?
- (4) If you needed to revise an earlier decision, how did you approach it in each condition?
- (5) If you had to extend this project with more features, which tool would you choose?

- (6) What did you like most about each approach?
- (7) What did you like least about each approach?

- (8) Any other thoughts or feedback?